Control de concurrencia en reportes

Documentación técnica del punto 2 de prioridad alta

Elemento	Información
Proyecto	City Care
Componente	Reportes e incidencias
Versión documentada	1.0
Fecha	2 de noviembre de 2026
Estado	Implementado y validado

Resumen

El punto 2 incorpora control de concurrencia optimista en los reportes. Cada reporte conserva una versión numérica y toda operación crítica indica la versión que el administrador tenía visible. Si el registro cambió antes de confirmar la operación, el backend rechaza la solicitud con HTTP 409 y evita sobrescribir información más reciente.

La solución cubre cambios de estado, asignaciones, reasignaciones, retiro de equipos y revisión de posibles duplicados. También actualiza el panel administrativo, el contrato OpenAPI y los clientes generados con Orval. La implementación quedó validada con compilación y 66 pruebas funcionales aprobadas.

1 Objetivo y alcance

El objetivo es impedir que dos administradores modifiquen el mismo reporte a partir de datos desactualizados. El mecanismo no bloquea la lectura ni mantiene bloqueos durante el tiempo que una persona observa la pantalla. La comprobación ocurre al guardar y se complementa con un bloqueo transaccional breve sobre la fila afectada.

- Evitar la pérdida silenciosa de cambios realizados por otro administrador.
- Mantener una única asignación activa por reporte.
- Conservar el historial de asignaciones y estados.
- Informar al usuario cuando su vista quedó desactualizada.
- Mantener sincronizados backend, OpenAPI, Orval y panel administrativo.

2 Problema que resuelve

Sin esta protección, dos administradores pueden abrir el mismo folio con los mismos datos. Si el primero asigna una cuadrilla y después el segundo guarda una asignación distinta desde la pantalla anterior, la segunda operación puede sustituir la primera sin advertencia. El mismo riesgo existe en los cambios de estado y en el retiro de una asignación.

Con el control implementado, la primera operación incrementa la versión. La segunda solicitud conserva la versión anterior, por lo que el backend detecta el conflicto, devuelve HTTP 409 y obliga a recargar el reporte antes de volver a decidir.

3 Diseño de la solución

1. El backend entrega el reporte con su campo version.
2. El panel conserva esa versión mientras el administrador revisa la información.
3. La operación de escritura envía expectedVersion junto con los demás datos.
4. La transacción bloquea la fila del reporte y compara ambas versiones.
5. Si coinciden, se guarda la operación y se incrementa version. Si no coinciden, se devuelve HTTP 409.

Concepto	Función
version	Versión actual almacenada en la tabla reports.
expectedVersion	Versión que el cliente tenía visible al iniciar la operación.
Bloqueo de fila	Serializa escrituras concurrentes dentro de una transacción corta.
HTTP 409	Indica que la operación entra en conflicto con el estado vigente.
Recarga	Actualiza el panel para que el administrador decida con información reciente.

4 Cambios en MySQL

Se agregó la migración 1796000000900_add_report_version_column. La migración modifica la tabla reports y crea una columna version de tipo entero sin signo, obligatoria y con valor inicial 1. Los reportes existentes reciben ese valor al aplicar la migración.

```
table.integer('version').unsigned().notNullable().defaultTo(1)
```

La migración se ejecutó en la base MySQL real y aparece con estado completed en el lote 10. No fue necesario crear una tabla adicional porque la versión pertenece al ciclo de vida del reporte.

5 Cambios en el backend

Componente	Responsabilidad
report_concurrency_service.ts	Compara expectedVersion y aumenta la versión del reporte.
report_queries_controller.ts	Protege cambios de estado y revisiones de duplicados.
work_teams_controller.ts	Protege asignaciones, reasignaciones y desasignaciones.
report_presenter_service.ts	Incluye version en las respuestas públicas y de detalle.
report.ts y work_team.ts	Exigen expectedVersion en las operaciones críticas.
handler.ts	Normaliza los mensajes de conflicto enviados a los clientes.

5.1 Servicio de concurrencia

La función assertReportVersion compara la versión bloqueada en MySQL con la versión enviada por el cliente. Si son distintas, lanza una excepción con estado 409 y código E_REPORT_CONCURRENCY_CONFLICT. La función incrementReportVersion aumenta la versión antes de guardar la operación aceptada.

```
if (report.version !== expectedVersion) {
  throw new Exception(
    'El reporte fue modificado por otro usuario. Recarga la información antes de continuar',
    { status: 409, code: 'E_REPORT_CONCURRENCY_CONFLICT' }
  )
}
```

5.2 Transacciones y bloqueo

Las escrituras consultan nuevamente el reporte dentro de una transacción y utilizan forUpdate. La comparación se realiza después de adquirir el bloqueo. Esta secuencia evita que dos solicitudes que llegan casi al mismo tiempo validen la misma versión y escriban ambas como si fueran la primera.

En una reasignación se cierran las asignaciones activas anteriores, se crea la nueva asignación, se actualiza currentAssignmentId y se incrementa version dentro de la misma transacción. Si cualquier paso falla, la transacción se revierte completa.

6 Operaciones protegidas

Operación	Método y ruta	Protección
Cambiar estado	PATCH /api/v1/admin/reports/{id}/status	Versión y bloqueo del reporte

Operación	Método y ruta	Protección
Asignar o reasignar	POST /api/v1/admin/reports/{id}/assignments	Versión, bloqueo y cierre de asignaciones activas
Retirar asignación	DELETE /api/v1/admin/reports/{id}/assignment	Versión y bloqueo del reporte
Revisar duplicado	PATCH /api/v1/admin/reports/{id}/possible-duplicates/{reviewId}	Bloqueo de revisión y estado pendiente

7 Contrato de la API

Las respuestas de listado y detalle incluyen version. Las solicitudes de cambio de estado, asignación y retiro requieren expectedVersion como entero positivo. El campo es obligatorio; una solicitud que no lo incluya se rechaza durante la validación.

7.1 Cambio de estado

```
{
  "status": "in_progress",
  "expectedVersion": 1,
  "note": "Cuadrilla revisando la incidencia"
}
```

7.2 Asignación de equipo

```
{
  "workTeamId": 8,
  "expectedVersion": 2,
  "priority": "high",
  "observations": "Atender antes del fin de semana"
}
```

7.3 Retiro de asignación

```
{
  "expectedVersion": 3,
  "note": "La incidencia será reasignada a otra cuadrilla"
}
```

7.4 Respuesta de conflicto

```
HTTP 409 Conflict

{
  "error": {
    "code": "E_REPORT_CONCURRENCY_CONFLICT",
    "message": "El reporte fue modificado por otro usuario. Recarga la información antes de
continuar"
  }
}
```

8 Integración con el panel administrativo

El panel de dependencia utiliza el valor version entregado por Orval. Antes de asignar o retirar un equipo busca el reporte visible y agrega expectedVersion al cuerpo de la solicitud. En el modal de detalle hace lo mismo al cambiar el estado.

Cuando recibe E_REPORT_CONCURRENCY_CONFLICT, el panel conserva el mensaje del backend y vuelve a consultar el listado. En un cambio de estado también recarga el detalle abierto. De esta forma el administrador ve el estado actual y puede repetir la acción de manera consciente.

La revisión de posibles duplicados utiliza E_DUPLICATE_REVIEW_CONFLICT. Si otra persona ya confirmó o descartó la coincidencia, el panel vuelve a consultar las revisiones y muestra el resultado vigente.

9 OpenAPI y Orval

El contrato OpenAPI fue regenerado desde el backend en http://127.0.0.1:3333/swagger y se copió al frontend. Después se regeneraron los clientes Orval de ambos proyectos. Los tipos ReportPublic y ReportDetailApiResponse ahora incluyen version; UpdateReportStatusRequest, AssignReportRequest y UnassignReportRequest incluyen expectedVersion.

Los archivos generados no deben modificarse manualmente. Cualquier cambio futuro debe comenzar en las interfaces, validadores y anotaciones del backend, seguido por la regeneración del contrato y de Orval.

10 Pruebas realizadas

Prueba	Resultado
Cambio de estado con versión vigente	Aceptado e incrementa la versión
Cambio de estado con versión antigua	HTTP 409 E_REPORT_CONCURRENCY_CONFLICT
Asignación con versión vigente	Aceptada y queda una asignación activa
Reasignación con versión antigua	HTTP 409 sin sobrescribir la asignación actual
Reasignación con versión vigente	Cierra la anterior y conserva el historial
Retiro de asignación	Libera el reporte e incrementa la versión
Segunda revisión de un duplicado	HTTP 409 E_DUPLICATE_REVIEW_CONFLICT
Suite funcional del backend	66 de 66 pruebas aprobadas
Typecheck y build del backend	Correctos
Typecheck y build del frontend	Correctos

11 Procedimiento de verificación manual

1. Iniciar el backend y el frontend con sus configuraciones habituales.
2. Abrir el mismo reporte en dos sesiones administrativas distintas.
3. En la primera sesión, asignar un equipo o cambiar el estado.
4. Sin recargar la segunda sesión, intentar modificar el mismo reporte.
5. Comprobar que la segunda solicitud recibe HTTP 409 y no sobrescribe el cambio anterior.
6. Confirmar que el panel recarga el reporte y muestra la versión vigente.
7. Consultar MySQL y verificar que version aumentó una sola vez por operación aceptada.

12 Consideraciones de mantenimiento

- Toda nueva operación que modifique un reporte debe exigir expectedVersion.
- La comparación debe ejecutarse dentro de la misma transacción que realiza la escritura.
- El reporte debe consultarse con bloqueo de fila antes de validar la versión.
- La versión solo debe aumentar cuando la operación completa se guarda correctamente.
- Los clientes deben tratar HTTP 409 como conflicto recuperable y recargar datos.
- No se debe reutilizar una versión recibida antes de una operación exitosa.

13 Criterios de aceptación

Criterio	Estado
La tabla reports contiene version con valor inicial 1	Cumplido
Las respuestas de reportes exponen version	Cumplido
Las escrituras críticas requieren expectedVersion	Cumplido
Las solicitudes desactualizadas reciben HTTP 409	Cumplido
Las operaciones aceptadas incrementan version	Cumplido
El panel recarga los datos después de un conflicto	Cumplido
OpenAPI y Orval representan el nuevo contrato	Cumplido
La suite completa permanece aprobada	Cumplido

Conclusión

El punto 2 de prioridad alta quedó implementado de extremo a extremo. MySQL conserva la versión, el backend valida y bloquea las operaciones críticas, el contrato comunica el requisito y el panel maneja los conflictos sin perder cambios. La solución mantiene los historiales existentes y ofrece una respuesta clara cuando una pantalla quedó desactualizada.